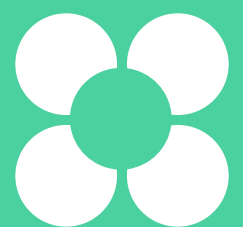


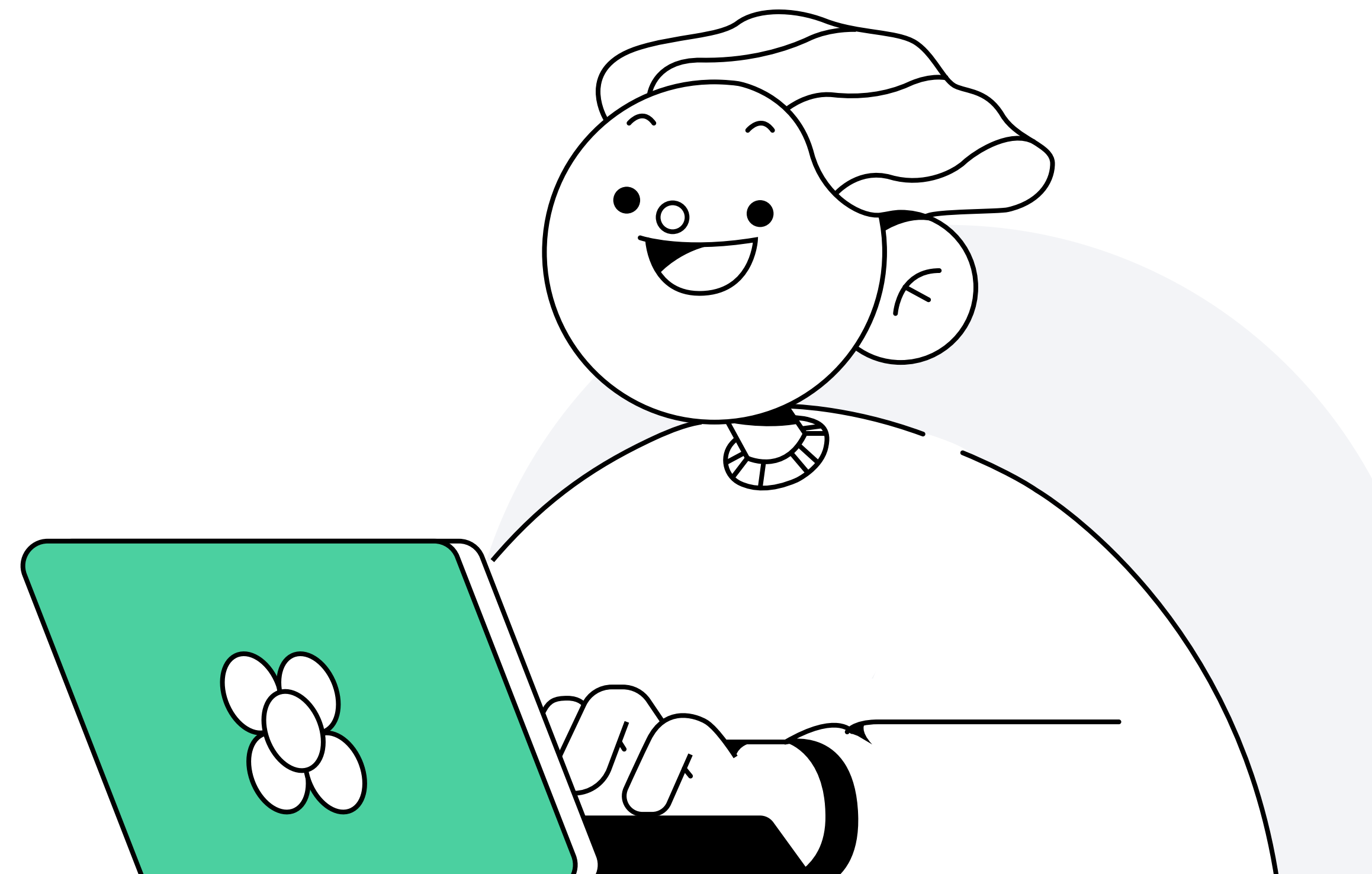
Абстракции и интерфейсы



План занятия

① Абстрактные классы

① Интерфейсы

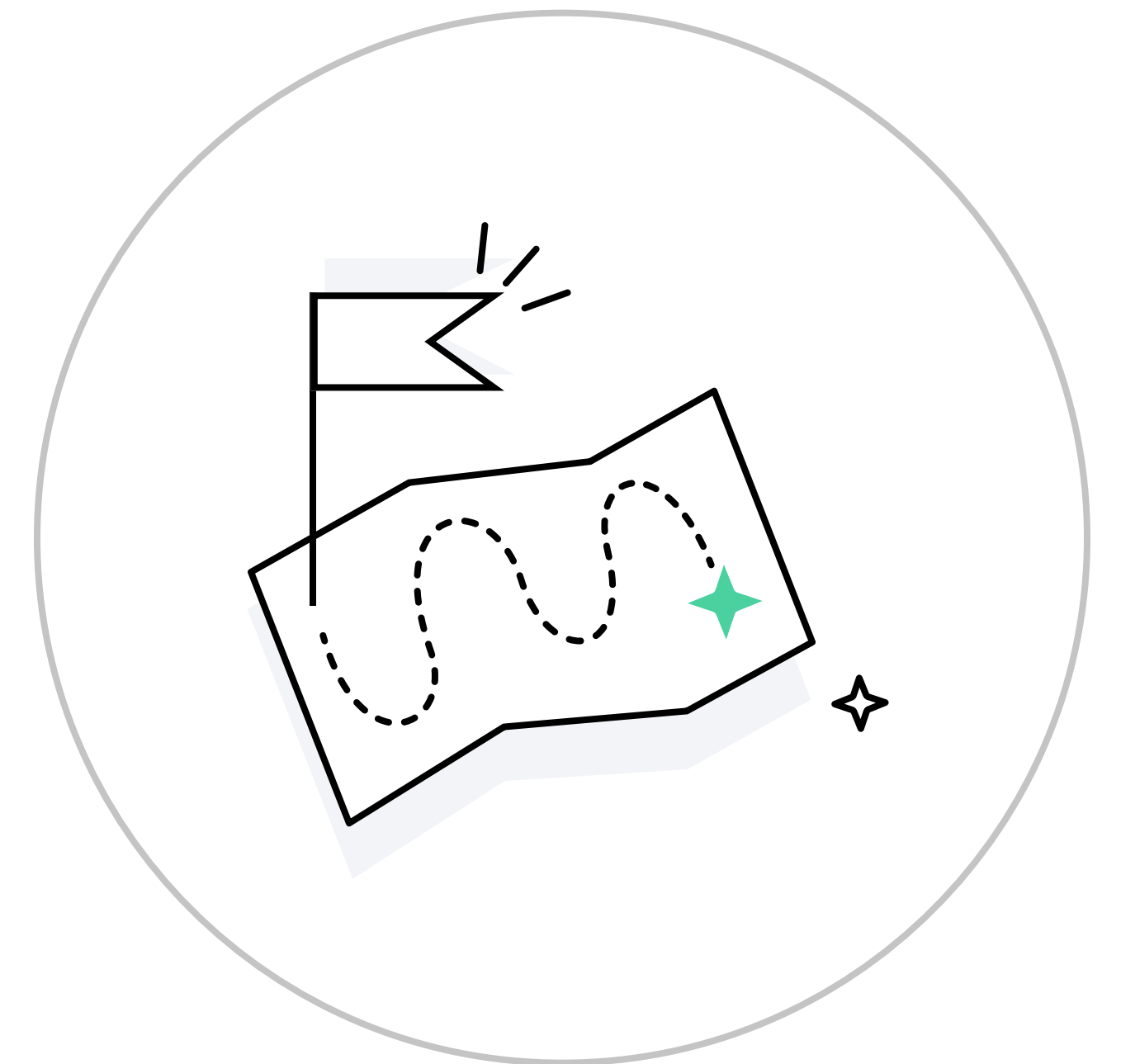


Абстрактные классы



Абстракция

Задача: перевезти пассажиров из одного места в другое с использованием разных видов транспорта



Абстракция

Для решения задачи создадим базовый класс **Transport**, который будет общим для всех видов транспорта:

```
public class Transport {  
    private int fuel;  
  
    public Transport(int fuel) {  
        this.fuel = fuel;  
    }  
  
    public void spendFuel(int fuel) {  
        this.fuel -= fuel;  
    }  
  
    public void transfer(String name, String from, String to) {}  
}
```

Абстракция

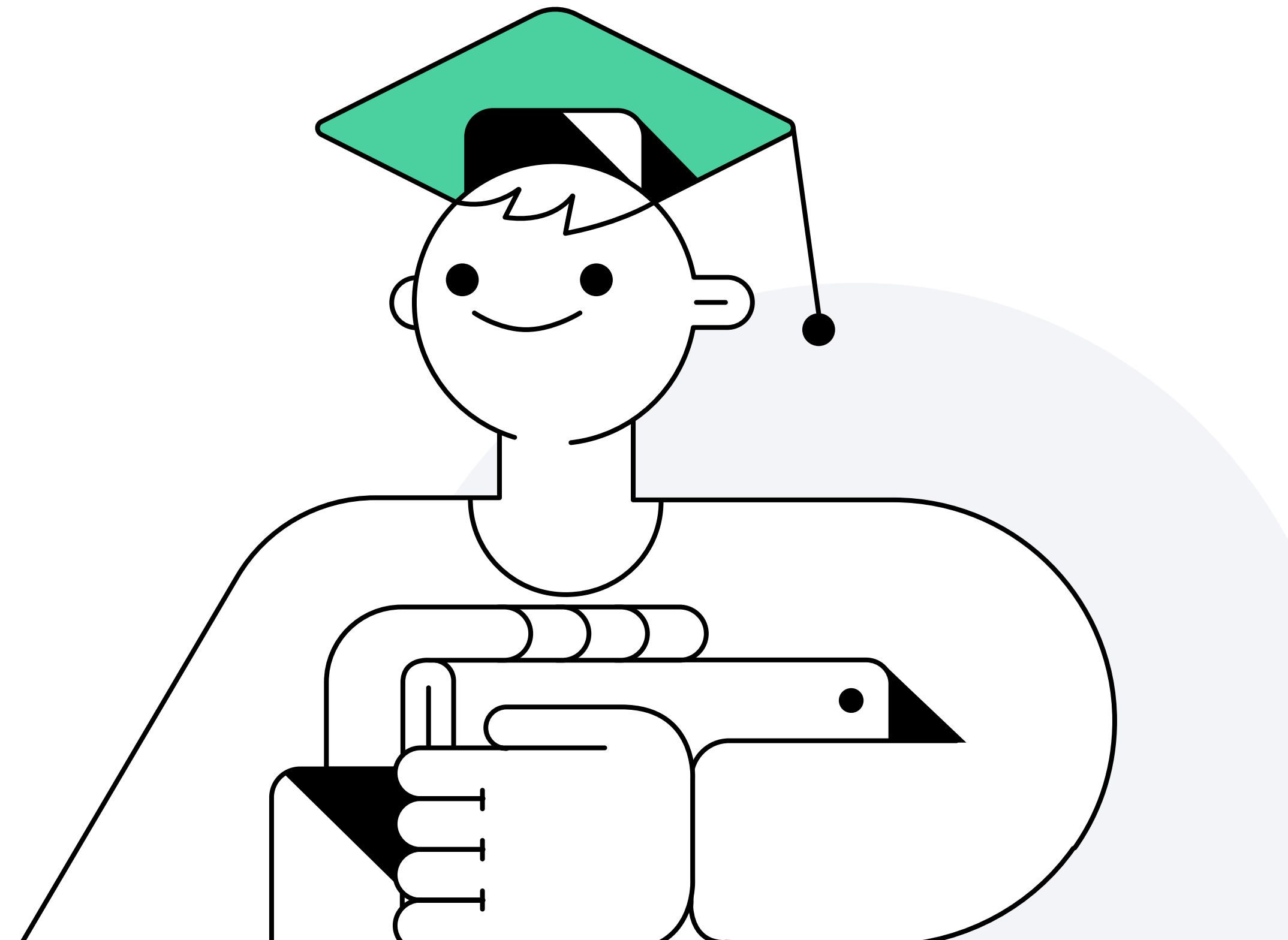
Создадим классы-наследники: Car, Ship

В каждом классе-наследнике создаём конструктор, так как в классе **Transport** есть конструктор с параметрами, и переопределяем метод перевозки пассажиров:

```
public class Car extends Transport {  
  
    public Car() {  
        super(100);  
    }  
  
    @Override  
    public void transfer(String name, String from, String to) {  
        System.out.println(name + " поехал на машине из " + from + " в " + to);  
        spendFuel(10);  
    }  
}
```


Задачи класса в ООП

- 1 Быть шаблоном для объектов
- 2 Быть основой для других классов через механизм наследования



Абстракция

При создании объектов на основе созданных классов мы можем столкнуться некоторыми проблемами:

- 1 Ничто не мешает нам создать объект базового класса **Transport**, а это нарушает философию объектно ориентированного программирования, поскольку этот класс был предназначен только для того, чтобы быть основой для других классов
- 2 Метод **transfer** в классе **Transport** был создан, чтобы работал полиморфизм, поэтому при вызове **transfer** для объекта **Transport** Java не выдаст нам ничего

Абстракция

- ③ Пользователь может забыть реализовать метод **transfer** для класса, который он создаёт, тогда опять же полиморфизм не сработает для этого класса

Решить эти проблемы можно с помощью **абстракции**.

Абстракция даёт нам возможность сказать Java, что базовый класс играет вторую роль, а не первую, и таким образом запретить Java создавать объект этого класса, заставить всех наследников давать конкретную реализацию методов и при этом сохранить свойства полиморфизма

Абстракция

Создадим абстрактный класс:

```
public abstract class Transport {  
    private int fuel;  
  
    public Transport(int fuel) {  
        this.fuel = fuel;  
    }  
  
    public void spendFuel(int fuel) {  
        this.fuel -= fuel;  
    }  
  
    public abstract void transfer(String name, String from, String to);  
}
```

Преимущества абстрактного класса

Абстрактный класс, как и обычный, содержит поля и методы.

Основные преимущества абстрактного класса:

- не может быть создан, а значит, не будет объектов с нереализованными методами
- обязывает потомков переопределить методы
- гарантирует наличие переопределённого метода в классе-потомке
- нет необходимости бросать исключения

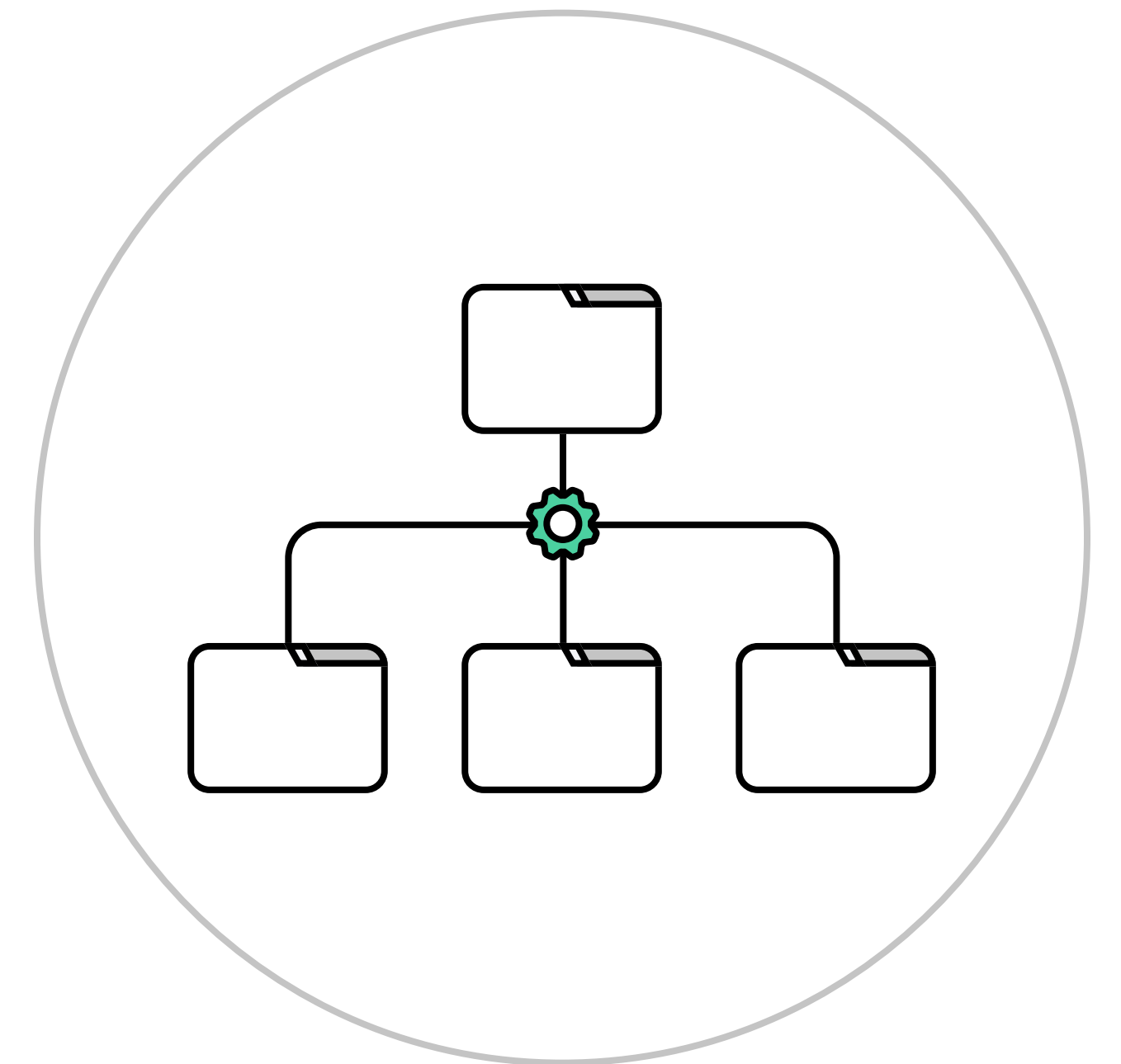
Интерфейсы



Интерфейсы

Предположим, что мы хотим создать ещё один вид транспорта — велосипед. Абстрактный класс `Transport` объединяет в себе виды транспорта, для передвижения которых необходимо топливо.

Велосипед же топливо не использует, следовательно, нам нужен ещё один уровень абстракции



Интерфейсы

Создадим абстрактный класс **Transferer**. И сделаем его базовым для класса **Transport**.

Класс **Transferer** будет настолько абстрактным, что у него не будет никаких членов класса (методов и полей): все они будут по умолчанию абстрактными, поэтому модификаторы **abstract** и **public** будут лишними.

Такого уровня абстракции в Java называются интерфейсами

```
public interface Transferer {  
    void transfer(String name, String from, String to);  
}
```


Интерфейсы

Если мы хотим, чтобы какой-то класс наследовался от интерфейса, то мы должны использовать слово **implements** вместо `extends`.

Любой наследник интерфейса получает в свой класс лишь набор абстрактных методов, которые он должен **реализовать**.

Создадим класс `Velo`:

```
public class Velo implements Transferer {  
  
    @Override  
    public void transfer(String name, String from, String to) {  
        System.out.println(name + " прокатился на велике из " + from + " в " + to);  
    }  
}
```

Интерфейсы

За счёт высокого уровня абстракции интерфейсы позволяют реализовать наследование класса от множества интерфейсов. В этом случае нам лишь необходимо перечислить наследуемые интерфейсы в **implements**:

```
public abstract class Transport implements Transferer, Billable {  
    //TODO  
}
```


Отличия абстрактного класса и интерфейса

Абстрактные классы:

- могут хранить состояние (поля) и описывать поведение (методы)
- описывают сходные по свойствам классы

Наследовать несколько абстрактных классов нельзя.

Интерфейсы:

- описывают только поведение (методы)
- могут быть реализованы на абсолютно разных классах

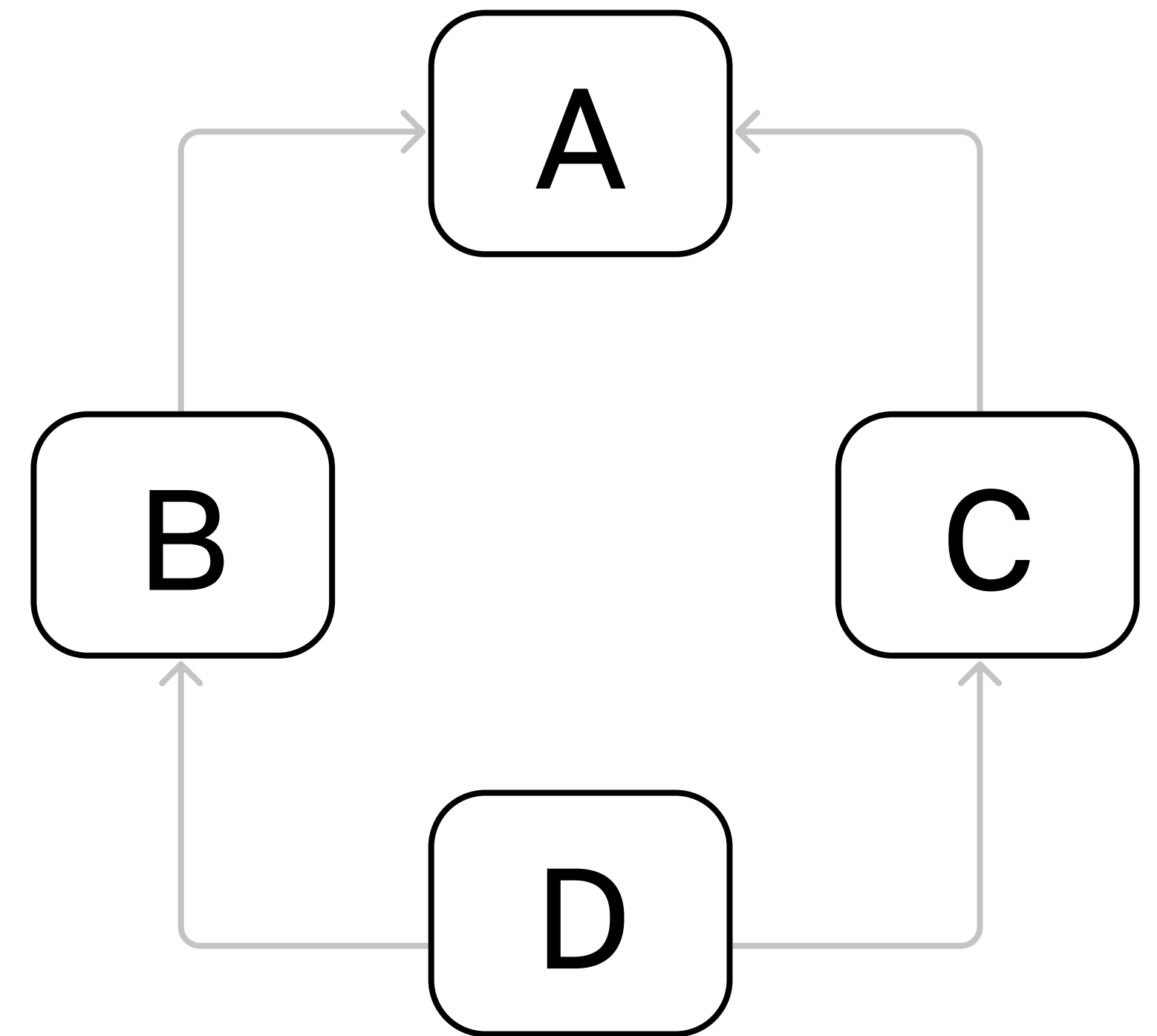
Класс может реализовывать несколько интерфейсов

Множественное наследование

Java-класс может наследоваться только от одного класса. Это сделано, чтобы избежать проблемы ромбовидного наследования.

Когда классы В и С наследуют класс А и класс D наследуется от них обоих, возникает проблема неопределённости: методы в В и С могут быть переопределены по-разному.

Считается, что если в вашей программе требуется множественное наследование, нужно пересмотреть архитектуру и, возможно, использовать интерфейсы



**Спасибо
за внимание!**

